

"User Journey & Application Flow Maps"

Document Type:	Application Flow Document
Version:	v1.0
Date:	June 2026
Author:	Platform Developer
Status:	Approved for Development

■ Table of Contents

- 1. Application Flow Overview
- 2. User Authentication Flow
- 3. Onboarding Flow
- 4. Mode 1: Idea Generator Flow
- 5. Mode 2: Evolution Engine Flow
- 6. Mode 3: Code Archaeology Flow
- 7. Deep Dive Flow (Blueprint + Mockup)
- 8. Dashboard & Project Management Flow
- 9. Export & Sharing Flow
- 10. Error Handling Flow
- 11. State Management Flow
- 12. Data Flow Diagrams

1. Application Flow Overview

IdeaDNA application flow is designed around three primary user modes, each with distinct input-processing-output cycles. All flows share common patterns: authentication, personalization context injection, lazy-loaded deep-dive generation, and persistent storage. This document maps every user action to system response with state transitions.

2. User Authentication Flow

- Step 1: User visits landing page (/) → Sees hero section + CTA buttons
- Step 2: User clicks 'Get Started' or 'Sign Up' → Redirect to /register
- Step 3: User selects auth method: Email/Password, Google OAuth, or GitHub OAuth
- Step 4A (Email): User enters email + password → Validation → Verification email sent → User clicks link → Account activated
- Step 4B (OAuth): User clicks provider → Redirect to provider auth → Callback to /api/auth/callback → Clerk creates session
- Step 5: Post-auth redirect to /onboarding (first time) or /dashboard (returning)
- Step 6: JWT access token stored in httpOnly cookie (7-day expiry)
- Step 7: Refresh token stored securely, rotated on every use
- Step 8: All subsequent requests include Authorization: Bearer header
- Step 9: Middleware validates token on every API call
- Step 10: Token expiry → Automatic refresh via refresh token → Seamless UX

3. Onboarding Flow

First-time user only — 6 questions, 2 minutes max

- Step 1: Welcome screen — 'Let's personalize your experience'
- Step 2: Q1 — Skill Level: Beginner / Intermediate / Advanced (radio buttons)
- Step 3: Q2 — Budget: \$0 / \$100-500 / \$500+ (radio buttons)
- Step 4: Q3 — Team Size: Just me / 2-3 people / 5+ people (radio buttons)
- Step 5: Q4 — Timeline: 2 weeks / 1 month / 3+ months (radio buttons)
- Step 6: Q5 — Tech Preferences: Multi-select (React, Node, Python, Flutter, etc.)
- Step 7: Q6 — Industry: Education, Healthcare, FinTech, etc. (dropdown, optional)
- Step 8: Review & Confirm — Summary of all selections
- Step 9: Save to database → Profile context ready for AI agents
- Step 10: Redirect to /dashboard with 'Your first project' prompt

4. Mode 1: Idea Generator Flow

Complete Flow: Problem → Ideas → Selection → Deep Dive

- Step 1: User clicks 'Generate Ideas' on dashboard → Navigate to /generate
- Step 2: Input screen shows: Platform selector (dropdown), Problem textarea, User profile summary (editable)
- Step 3: User selects platform (e.g., 'Web App') + describes problem (e.g., 'Students struggle to find internships')
- Step 4: User clicks 'Generate' → Frontend shows loading: 'Analyzing your problem...' with DNA helix animation
- Step 5: API call: POST /api/projects (create project) → POST /api/projects/:id/generate (trigger generation)
- Step 6: Backend: InputParserAgent → RetrieverAgent (semantic search curated DB) → UpgradeAgent (personalize) → ScorerAgent (rank)
- Step 7: Response: 5-8 idea cards with title, description, innovation score, feasibility score, difficulty, tags
- Step 8: Frontend renders cards in responsive grid (Desktop: 3 cols, Tablet: 2 cols, Mobile: 1 col)
- Step 9: User hovers card → Lift animation + glow effect + 'Select' button appears
- Step 10: User clicks 'Select' on one card → API call: POST /api/ideas/:id/select
- Step 11: Backend: Status changes to SELECTED, triggers BlueprintArchitectAgent + UIDesignerAgent (async)
- Step 12: Frontend: Selected card expands, shows loading state 'Generating your blueprint...'
- Step 13: Backend completes → Saves blueprint + mockup to DB
- Step 14: Frontend auto-refreshes → Shows Blueprint tab + UI Mockup tab
- Step 15: User explores: Overview → PRD → Tech Stack → Database → API → User Flow → Timeline (accordion)
- Step 16: User switches to UI Mockup tab → Iframe renders live HTML → Device toggle available
- Step 17: User actions: Export (Markdown/PDF/HTML), Evolve Further, Save Project, Start New

5. Mode 2: Evolution Engine Flow

Complete Flow: Existing Idea → Critic → Evolution Tree → Selection → Deep Dive

- Step 1: User clicks 'Evolve Idea' on dashboard → Navigate to /evolve
- Step 2: Input screen: Title field + Description textarea + Current features (optional)
- Step 3: User clicks 'Analyze & Evolve' → Loading: 'Analyzing weaknesses...'
- Step 4: API call: POST /api/projects → POST /api/projects/:id/evolve
- Step 5: Backend: CriticAgent analyzes idea → Returns weakness report (visible to user)
- Step 6: Frontend shows Critic Report: Weaknesses (HIGH/MEDIUM/LOW), Gaps, Risks, Opportunities
- Step 7: User clicks 'Generate Evolutions' → Loading: 'Creating mutations...'
- Step 8: Backend: EvolverAgent generates 4-6 versions → ScorerAgent scores each
- Step 9: Frontend renders React Flow evolution tree: Root = Original Idea, Branches = Evolved Versions
- Step 10: Tree features: Click node → Details panel, Hover edge → Mutation tag, Color = score-based
- Step 11: User clicks one version node → 'Select This Version' button
- Step 12: Selection triggers same deep-dive as Generator mode (Blueprint + Mockup)
- Step 13: User can click 'Evolve Further' on any selected version → Recursive evolution
- Step 14: New tree generated with selected version as root → Infinite depth possible

6. Mode 3: Code Archaeology Flow

Complete Flow: Code Input → Parse → Analyze → Report → Evolution

- Step 1: User clicks 'Analyze Code' on dashboard → Navigate to /analyze
- Step 2: Input options: Tab 1 = Paste Code, Tab 2 = GitHub URL, Tab 3 = Website URL
- Step 3A (Paste): Textarea for code input + Language auto-detect dropdown
- Step 3B (GitHub): URL input + OAuth connect (for private repos)
- Step 3C (Website): URL input + Disclaimer about scraping permissions
- Step 4: User clicks 'Analyze' → Loading: 'Excavating code...' with archaeology animation
- Step 5: API call: POST /api/projects → POST /api/projects/:id/analyze
- Step 6: Backend 6-Agent Pipeline: CodeParser → PatternDetector → IdeaHistorian → DevPsychologist → Scorer → Evolver
- Step 7: Frontend shows progress: Step 1/6, Step 2/6... with agent names
- Step 8: Complete report rendered in sections: Code Analysis, Patterns, Evolution Timeline, Developer Psychology, Scores, Future Versions
- Step 9: Each section: Collapsible panel with structured data + visual indicators
- Step 10: User can: Export report as PDF, Evolve this idea (creates new project), Save to library

7. Deep Dive Flow (Blueprint + Mockup)

Triggered ONLY after idea selection — Lazy-loaded

- Step 1: User selects idea → Status = SELECTED → Deep-dive unlocks
- Step 2: Two tabs appear: 'Blueprint' (default) and 'UI Mockup'
- Step 3A (Blueprint Tab): 7 accordion sections — Overview, PRD, Tech Stack, Database Schema, API Design, User Flow, Implementation Plan
- Step 3B: Each section: Expand/collapse, Copy button, Export individual section
- Step 4: Full Export buttons: Markdown, PDF, OpenAPI YAML
- Step 5: User switches to 'UI Mockup' tab → Iframe loads with HTML preview
- Step 6: Device toggle: Desktop (100%), Tablet (768px), Mobile (375px)
- Step 7: Mockup features: Scrollable, Interactive elements (buttons hover), Real content
- Step 8: Style selector: Modern, Minimal, Cyberpunk, Corporate, Playful → Regenerate
- Step 9: Export options: HTML file, ZIP (HTML + CSS + assets)
- Step 10: User can switch between Blueprint and Mockup anytime

8. Dashboard & Project Management Flow

- Step 1: User navigates to /dashboard → Shows all projects
- Step 2: Layout: Sidebar (filters) + Main area (project cards)
- Step 3: Filters: Mode (Generate/Evolve/Analyze), Status (Draft/Selected/Completed), Date range, Platform, Tags
- Step 4: Search bar: Text search + semantic search toggle
- Step 5: Project cards: Title, mode badge, status, created date, overall score, quick actions
- Step 6: Quick actions: Continue (opens project), Evolve (creates evolution), Export, Delete, Share
- Step 7: Stats section: Total projects, ideas generated, average score, favorite platform
- Step 8: 'Similar Projects' section: Vector search results with similarity score
- Step 9: 'Recent Activity' timeline: Last 10 actions with timestamps
- Step 10: 'Quick Start' buttons: New Problem, Evolve Existing, Analyze Code

9. Export & Sharing Flow

- Step 1: User clicks 'Export' on any project/idea/blueprint/mockup
- Step 2: Modal opens with format options based on content type
- Step 3A (Blueprint): Markdown, PDF, OpenAPI YAML, Mermaid diagrams
- Step 3B (Mockup): HTML file, ZIP package
- Step 3C (Project): ZIP with all data + assets
- Step 4: User selects format → System generates file (async if large)
- Step 5: Download starts automatically or email sent (for large files)
- Step 6: Share: Toggle 'Make Public' → Generate unique URL → Copy to clipboard
- Step 7: Shared URL: View-only mode, no editing, shows project summary
- Step 8: Revoke sharing: Toggle off → URL invalidated immediately

10. Error Handling Flow

- Validation Error (400): Field-level red borders + inline messages + form scroll to first error
- Auth Error (401): Redirect to login with 'Session expired' toast + return URL preserved
- Forbidden (403): 'Unauthorized' page with contact support link
- Not Found (404): Custom 404 page with 'Go to Dashboard' CTA
- Rate Limit (429): Toast with countdown timer + 'Upgrade plan' CTA
- Server Error (500): Generic 'Something went wrong' + 'Retry' button + auto-report to Sentry
- AI API Error (502/503): 'AI service temporarily unavailable' + cached fallback if available + retry button
- Network Error: 'Connection lost' banner + queue actions for sync when online
- Timeout Error: 'Request took too long' + partial results if available + retry

11. State Management Flow

Zustand (Client State) + React Query (Server State)

- Client State (Zustand): Auth user, UI preferences (theme, sidebar), Active project ID, Selected idea ID, Modal states, Toast notifications
- Server State (React Query): Projects list (staleTime: 5min), Current project (staleTime: 1min), Ideas (staleTime: 2min), Blueprint (staleTime: 10min), Mockup (staleTime: 10min), User profile (staleTime: 30min)
- Cache Invalidation: On mutation success → invalidate related queries, On logout → clear all queries, On project delete → remove from cache
- Optimistic Updates: Idea selection (UI updates immediately, rollback on error), Project creation (add to list immediately), Profile update (reflect changes immediately)

12. Data Flow Diagrams

12.1 Idea Generation Data Flow

→ User Input → Frontend Validation (Zod) → API Gateway (Next.js Route)
→ → Auth Middleware (Clerk JWT validation)
→ → Rate Limiting Check (Redis)
→ → IdeaGeneratorService
→ → InputParserAgent (GPT-4, temp 0.3) → Parsed Input JSON
→ → RetrieverAgent → PostgreSQL (pgvector semantic search) → Top 10 matches
→ → UpgradeAgent (GPT-4, temp 0.6) → Personalized ideas
→ → ScorerAgent (GPT-4, temp 0.2) → Scored ideas
→ → Save to DB (Project + Ideas)
→ → Cache in Redis (5 min)
→ → Response to Frontend → Render cards

12.2 Deep Dive (Lazy Load) Data Flow

→ User Selects Idea → API call: POST /api/ideas/:id/select
→ → Update status = SELECTED in DB
→ → Trigger BlueprintArchitectAgent (async, background)
→ → Trigger UIDesignerAgent (async, background)
→ → Frontend shows loading state
→ → Agents complete → Save Blueprint + Mockup to DB
→ → Frontend auto-refreshes (React Query polling)
→ → Render Blueprint tabs + Mockup preview

— End of APPFLOW —